

Estructuras de datos básicas

Notas de clase

Estructuras de datos y algoritmos

Facultad TIC - UPB

Spring 2026

Jorge Mario Londoño Peláez

8 de abril de 2026

Basic Data Structures

En este capítulo se estudian las estructuras de datos básicas: bolsas, pilas, colas y listas enlazadas.



**Universidad
Pontificia
Bolivariana**

Índice

1. Estructuras de datos básicas	2
1.1. Bolsa (Bag)	2
1.2. Pila (Stack)	2
1.3. Cola (Queue)	3
1.4. Ejemplo de implementación de la Pila basada en un arreglo	3
1.5. Tipos genéricos	5
1.6. Pila con arreglo de tamaño dinámico (Resizing Array)	7
1.7. Tipos wrapper	8
1.8. Iteradores	9
1.8.1. Iteradores en Java	9
1.8.2. Iteradores en Python	10
1.8.3. Iteradores en C#	11
1.9. Implementación de la Cola con Arreglo Circular	14
1.10. Implementación de la Bolsa con Arreglo	16
1.11. Resumen comparativo	18
1.12. De arreglos a listas enlazadas	18
2. Listas Simplemente Enlazadas	19
2.1. Definición recursiva de lista	19
2.2. Implementación de la lista simplemente enlazada	19
2.2.1. Definición de la clase Nodo	19
2.2.2. Construcción de una lista paso a paso	19
2.2.3. Implementación de la lista enlazada como ADT	20
2.3. Otras operaciones con listas simplemente enlazadas	22
2.3.1. Eliminar el primer elemento de la lista	22
2.3.2. Agregar un elemento al final de la lista	23
2.3.3. Eliminar el último elemento de la lista	23
2.3.4. Otras operaciones con listas simples	24
2.4. Iteradores sobre listas simplemente enlazadas	25
2.4.1. Proceso de recorrido de una lista	25
2.4.2. Operaciones que dependen de un recorrido de la lista	25
2.5. Resumen de complejidad de operaciones	28
3. Listas Doblemente Enlazadas	28
3.1. ADT e Implementación de la lista doblemente enlazada	29
3.2. La cola de doble terminación (DEQUE)	31

1. Estructuras de datos básicas

En esta sección, exploraremos las estructuras de datos básicas, diferenciándolas de estructuras más simples como arreglos y matrices. Mientras que un arreglo tiene un tamaño fijo y almacena elementos de un único tipo predefinido, las estructuras que estudiaremos ofrecen mayor flexibilidad y adaptabilidad en la gestión de datos.

En particular, buscaremos que nuestras implementaciones cumplan tres objetivos:

1. Ser **dinámicas**, es decir, que puedan crecer o decrecer según la necesidad.
2. Garantizar la **consistencia de tipos**, para evitar errores al mezclar datos de tipos diferentes.
3. Permitir **recorrer sus elementos** de forma estándar mediante iteradores.

El capítulo se organiza de la siguiente manera: primero, definiremos las tres estructuras básicas (Bolsa, Pila y Cola) a nivel de Application Programming Interface (API). Luego, implementaremos la Pila basada en un arreglo para exponer las limitaciones de un enfoque simple. A continuación, abordaremos las técnicas que resuelven esas limitaciones: tipos genéricos, arreglos de tamaño dinámico, tipos wrapper e iteradores. Finalmente, presentaremos implementaciones completas de la Cola y la Bolsa aplicando todas estas técnicas. En la segunda parte del capítulo, estudiaremos las listas enlazadas como mecanismo alternativo de almacenamiento.

1.1. Bolsa (Bag)

Una **Bolsa** es una colección no ordenada de elementos, donde se permite la duplicación. Es útil cuando se necesita almacenar elementos sin importar el orden y sin necesidad de acceder a ellos por una posición específica.

API de la Bolsa:

- `add(element)`: Agrega un elemento a la bolsa.
- `remove(element)`: Elimina una instancia del elemento de la bolsa (si existe).
- `isEmpty()`: Verifica si la bolsa está vacía.
- `size()`: Retorna el número de elementos en la bolsa.
- `contains(element)`: Verifica si la bolsa contiene el elemento.
- `iterator()`: Retorna un iterador para recorrer los elementos de la bolsa.

Ejemplo de aplicación: Una bolsa puede ser usada para almacenar los productos en un carrito de compras, donde el orden de los productos no es relevante y puede haber duplicados.

1.2. Pila (Stack)

Una **Pila** es una colección ordenada de elementos que sigue el principio Last In, First Out (LIFO), es decir, el último elemento en entrar es el primero en salir. Es útil para gestionar el orden de ejecución de funciones, el historial de navegación, etc.

API de la Pila:

- `push(element)`: Agrega un elemento a la cima de la pila.
- `pop()`: Elimina y retorna el elemento de la cima de la pila.

- `peek()`: Retorna el elemento de la cima de la pila sin eliminarlo.
- `isEmpty()`: Verifica si la pila está vacía.
- `size()`: Retorna el número de elementos en la pila.

Ejemplo de aplicación: Una pila puede ser usada para implementar la funcionalidad de "deshacer" en un editor de texto, donde la última acción realizada es la primera en ser deshecha.

1.3. Cola (Queue)

Una **Cola** es una colección ordenada de elementos que sigue el principio First In, First Out (FIFO), es decir, el primer elemento en entrar es el primero en salir. Es útil para gestionar tareas en espera, simular filas de atención, etc.

API de la Cola:

- `enqueue(element)`: Agrega un elemento al final de la cola.
- `dequeue()`: Elimina y retorna el elemento del frente de la cola.
- `peek()`: Retorna el elemento del frente de la cola sin eliminarlo.
- `isEmpty()`: Verifica si la cola está vacía.
- `size()`: Retorna el número de elementos en la cola.

Ejemplo de aplicación: Una cola puede ser usada para gestionar la cola de impresión de documentos, donde el primer documento enviado a imprimir es el primero en ser impreso.

1.4. Ejemplo de implementación de la Pila basada en un arreglo

A continuación, se presenta una implementación de la Pila basada en un arreglo en Java. Esta implementación utiliza un arreglo de `String` para almacenar los elementos de la pila.

```
1 public class StackArray {
2     private String[] stack;
3     private int top;
4     private int capacity;
5
6     public StackArray(int capacity) {
7         this.capacity = capacity;
8         this.stack = new String[capacity];
9         this.top = -1;
10    }
11
12    public void push(String element) {
13        if (top == capacity - 1) {
14            throw new IllegalStateException("Stack is full");
15        }
16        stack[++top] = element;
17    }
18
19    public String pop() {
20        if (isEmpty()) {
21            throw new IllegalStateException("Stack is empty");
22        }
23        return stack[top--];
24    }
25 }
```

```
24     }
25
26     public String peek() {
27         if (isEmpty()) {
28             throw new IllegalStateException("Stack is empty");
29         }
30         return stack[top];
31     }
32
33     public boolean isEmpty() {
34         return top == -1;
35     }
36
37     public int size() {
38         return top + 1;
39     }
40 }
```

Listing 1: Implementación de la Pila basada en un arreglo en Java

Ejemplo de pruebas unitarias para los métodos de la Pila:

```
1 public class StackArrayTest {
2     public static void main(String[] args) {
3         StackArray stack = new StackArray(5);
4         stack.push("A");
5         stack.push("B");
6         stack.push("C");
7
8         assert stack.size() == 3;
9         assert stack.peek().equals("C");
10        assert stack.pop().equals("C");
11        assert stack.size() == 2;
12        assert !stack.isEmpty();
13
14        assert stack.pop().equals("B");
15        assert stack.pop().equals("A");
16        assert stack.isEmpty();
17    }
18 }
```

Listing 2: Ejemplo de pruebas unitarias para la Pila

Nota: Para que las aserciones (`assert`) funcionen, es necesario habilitarlas al ejecutar el programa desde la línea de comandos, usando el flag `-ea` (enable assertions). Ejemplo: `java -ea StackArrayTest`. Sin este flag, las líneas con `assert` serán ignoradas.

Discusión sobre implementación de otras estructuras: Las tres estructuras (Pila, Cola y Bolsa) comparten características comunes pero difieren en sus políticas de inserción y eliminación. En la Bolsa, la inserción se hace al final del arreglo y la eliminación requiere buscar el elemento (no hay orden específico). En la Cola, la inserción se hace al final y la eliminación al inicio (FIFO), lo que requiere técnicas especiales como el arreglo circular que veremos más adelante para evitar el costo de mover elementos.

Limitaciones de esta implementación: Esta implementación tiene las siguientes limitaciones:

- **Tamaño fijo:** El tamaño de la pila se define al momento de la creación y no puede cambiar.
- **Solo soporta un tipo:** La pila solo puede almacenar elementos de tipo `String`.

Para solucionar estas limitaciones, se pueden usar tipos genéricos y arreglos de tamaño dinámico. Estas limitaciones no son exclusivas de la Pila: afectan a **todas** las estructuras basadas en arreglos. En las siguientes secciones abordaremos estas técnicas en el contexto de la Pila, pero las mismas soluciones se aplican directamente a la Cola y la Bolsa, como veremos más adelante.

1.5. Tipos genéricos

En la implementación anterior, la pila solo podía almacenar cadenas de texto (**String**). Si quisiéramos una pila de enteros o de objetos de otra clase, tendríamos que duplicar todo el código. Los **tipos genéricos** resuelven este problema: permiten definir clases e interfaces que trabajan con diferentes tipos de datos sin perder la verificación de tipos en tiempo de compilación. Esto significa que se puede escribir código que funcione con cualquier tipo de dato, sin necesidad de escribir código específico para cada tipo.

Características de los tipos genéricos:

- **Reutilización de código:** Se puede escribir una sola clase o interfaz que funcione con diferentes tipos de datos.
- **Seguridad de tipos:** El compilador verifica que los tipos de datos utilizados sean correctos, evitando errores en tiempo de ejecución.
- **Eliminación de casting:** No es necesario realizar conversiones de tipo explícitas (casting), ya que el compilador conoce el tipo de dato con el que se está trabajando.

Ejemplo de implementación de la Pila basada en arreglo con tipos genéricos en Java: A continuación, se presenta una implementación de la Pila basada en un arreglo en Java, utilizando tipos genéricos. Esta implementación puede almacenar elementos de cualquier tipo.

```
1 public class StackGeneric<T> {
2     private T[] stack;
3     private int top;
4     private int capacity;
5
6     public StackGeneric(int capacity) {
7         this.capacity = capacity;
8         this.stack = (T[]) new Object[capacity];
9         this.top = -1;
10    }
11
12    public void push(T element) {
13        if (top == capacity - 1) {
14            throw new IllegalStateException("Stack is full");
15        }
16        stack[++top] = element;
17    }
18
19    public T pop() {
20        if (isEmpty()) {
21            throw new IllegalStateException("Stack is empty");
22        }
23        T element = stack[top];
24        stack[top] = null; // Evitar loitering
25        top--;
26        return element;
27    }
28 }
```

```
28
29     public T peek() {
30         if (isEmpty()) {
31             throw new IllegalStateException("Stack is empty");
32         }
33         return stack[top];
34     }
35
36     public boolean isEmpty() {
37         return top == -1;
38     }
39
40     public int size() {
41         return top + 1;
42     }
43 }
```

Listing 3: Implementación de la Pila basada en un arreglo con tipos genéricos en Java

Nota sobre *loitering*: Al eliminar un elemento de la pila, se asigna `null` a la posición correspondiente del arreglo. Esto se conoce como evitar *loitering*: sin esta asignación, el arreglo mantendría una referencia al objeto eliminado, impidiendo que el recolector de basura (*garbage collector*) libere su memoria, incluso si el objeto ya no es accesible a través de la pila.

Nota sobre la creación del arreglo genérico: En Java no es posible crear un arreglo de un tipo genérico directamente (es decir, `new T[capacity]` no compila) debido al borrado de tipos (*type erasure*). La alternativa estándar es crear un arreglo de `Object` y hacer un *cast* explícito: `(T[]) new Object[capacity]`. Este *cast* produce una advertencia del compilador (*unchecked cast*), la cual puede suprimirse con la anotación `@SuppressWarnings("unchecked")` si se tiene la certeza de que el arreglo solo contendrá elementos de tipo `T`.

Ejemplo de pruebas unitarias para la Pila genérica:

```
1 public class StackGenericTest {
2     public static void main(String[] args) {
3         StackGeneric<Integer> stack = new StackGeneric<>(5);
4         stack.push(1);
5         stack.push(2);
6         stack.push(3);
7
8         assert stack.size() == 3;
9         assert stack.peek().equals(3);
10        assert stack.pop().equals(3);
11        assert stack.size() == 2;
12        assert !stack.isEmpty();
13
14        assert stack.pop().equals(2);
15        assert stack.pop().equals(1);
16        assert stack.isEmpty();
17
18        StackGeneric<String> stringStack = new StackGeneric<>(5);
19        stringStack.push("A");
20        stringStack.push("B");
21        stringStack.push("C");
22
23        assert stringStack.size() == 3;
24        assert stringStack.peek().equals("C");
25        assert stringStack.pop().equals("C");
26        assert stringStack.size() == 2;
27        assert !stringStack.isEmpty();
28    }
29 }
```

```
28
29     assert stringStack.pop().equals("B");
30     assert stringStack.pop().equals("A");
31     assert stringStack.isEmpty();
32 }
33 }
```

Listing 4: Ejemplo de pruebas unitarias para la Pila genérica

1.6. Pila con arreglo de tamaño dinámico (Resizing Array)

La implementación anterior tiene una limitación importante: un tamaño fijo. Para solucionar esto, podemos usar una estrategia de redimensionamiento dinámico. La idea es empezar con un arreglo de tamaño pequeño y, cuando se llene, crear un nuevo arreglo más grande (normalmente el doble de tamaño) y copiar los elementos. De manera similar, para no desperdiciar memoria, si al eliminar elementos el arreglo queda muy vacío (por ejemplo, a un cuarto de su capacidad), lo redimensionamos a un tamaño más pequeño (la mitad).

Esta estrategia asegura que el costo de cada operación (push y pop), en promedio, sea constante (costo amortizado $O(1)$).

```
1 public class StackResizingArray<T> {
2     private T[] stack;
3     private int n; // Numero de elementos en la pila
4
5     public StackResizingArray() {
6         stack = (T[]) new Object[2]; // Empezar con capacidad 2
7         n = 0;
8     }
9
10    public boolean isEmpty() {
11        return n == 0;
12    }
13
14    public int size() {
15        return n;
16    }
17
18    private void resize(int capacity) {
19        assert capacity >= n;
20        T[] copy = (T[]) new Object[capacity];
21        for (int i = 0; i < n; i++) {
22            copy[i] = stack[i];
23        }
24        stack = copy;
25    }
26
27    public void push(T element) {
28        if (n == stack.length) {
29            resize(2 * stack.length); // Doblar el tamaño si esta lleno
30        }
31        stack[n++] = element;
32    }
33
34    public T pop() {
35        if (isEmpty()) {
36            throw new IllegalStateException("Stack is empty");
37        }
38    }
39 }
```



```

38     T element = stack[--n];
39     stack[n] = null; // Evitar "loitering" (mantener referencias a objetos no
    usados)
40     if (n > 0 && n == stack.length / 4) {
41         resize(stack.length / 2); // Reducir a la mitad si esta a 1/4 de
    capacidad
42     }
43     return element;
44 }
45
46 public T peek() {
47     if (isEmpty()) {
48         throw new IllegalStateException("Stack is empty");
49     }
50     return stack[n - 1];
51 }
52 }

```

Listing 5: Implementación de la Pila con arreglo dinámico en Java

1.7. Tipos wrapper

En Java, existen dos categorías principales de tipos de datos: **tipos primitivos** y **tipos wrapper**. Los tipos primitivos son los tipos de datos básicos, como `int`, `double`, `boolean`, etc. Los tipos wrapper, por otro lado, son clases que *envuelven*^a los tipos primitivos, como `Integer`, `Double`, `Boolean`, etc.

Diferencias entre tipos primitivos y tipos wrapper:

- **Almacenamiento:** Los tipos primitivos almacenan directamente el valor, mientras que los tipos wrapper almacenan una referencia a un objeto que contiene el valor.
- **Valores nulos:** Los tipos primitivos no pueden ser nulos, mientras que los tipos wrapper sí pueden serlo.
- **Métodos:** Los tipos wrapper ofrecen métodos útiles para trabajar con los valores que contienen, como convertir a `String`, comparar, etc.

Uso de tipos wrapper en genéricos: En Java, los tipos genéricos solo pueden trabajar con tipos de referencia, es decir, con objetos. Por lo tanto, no se pueden usar tipos primitivos directamente en estructuras genéricas como `StackGeneric<int>`. En su lugar, se deben usar los tipos wrapper correspondientes, como `StackGeneric<Integer>`.

Autoboxing y unboxing: Java ofrece la funcionalidad de **autoboxing** y **unboxing** para facilitar el trabajo con tipos primitivos y wrapper.

- **Autoboxing:** Es la conversión automática de un tipo primitivo a su tipo wrapper correspondiente. Por ejemplo, `Integer i = 5;` es un ejemplo de autoboxing, donde el valor 5 de tipo `int` es convertido automáticamente a un objeto de tipo `Integer`.
- **Unboxing:** Es la conversión automática de un tipo wrapper a su tipo primitivo correspondiente. Por ejemplo, `int j = i;` es un ejemplo de unboxing, donde el objeto `i` de tipo `Integer` es convertido automáticamente a un valor de tipo `int`.

Gracias al autoboxing y unboxing, se puede trabajar con tipos primitivos y wrapper de forma transparente al implementar estructuras genéricas. Por ejemplo, se puede usar `stack.push(5);` en una pila de tipo `StackGeneric<Integer>`, y Java se encargará de convertir el valor 5 de tipo `int` a un objeto de tipo `Integer` antes de agregarlo a la pila.

1.8. Iteradores

Un **iterador** es un objeto que permite recorrer los elementos de una colección, uno a la vez, sin necesidad de conocer la estructura interna de la colección. Los iteradores proporcionan una forma estándar de acceder a los elementos de una colección, independientemente de cómo estén almacenados. Esto permite que el código que usa la colección sea más genérico y reutilizable.

1.8.1. Iteradores en Java

En Java, los iteradores se implementan utilizando las interfaces `Iterable` e `Iterator`. La interfaz `Iterable` define el método `iterator()`, que retorna un objeto de tipo `Iterator`. La interfaz `Iterator` define los métodos `hasNext()`, que verifica si hay más elementos en la colección, y `next()`, que retorna el siguiente elemento de la colección.

A continuación, se presenta un ejemplo de implementación del iterador para la clase `StackGeneric` en Java:

```
1 import java.util.Iterator;
2 import java.util.NoSuchElementException;
3
4 public class StackGeneric<T> implements Iterable<T> {
5     private T[] stack;
6     private int top;
7     private int capacity;
8
9     public StackGeneric(int capacity) {
10         this.capacity = capacity;
11         this.stack = (T[]) new Object[capacity];
12         this.top = -1;
13     }
14
15     public void push(T element) {
16         if (top == capacity - 1) {
17             throw new IllegalStateException("Stack is full");
18         }
19         stack[++top] = element;
20     }
21
22     public T pop() {
23         if (isEmpty()) {
24             throw new IllegalStateException("Stack is empty");
25         }
26         T element = stack[top];
27         stack[top] = null; // Evitar loitering
28         top--;
29         return element;
30     }
31
32     public T peek() {
33         if (isEmpty()) {
34             throw new IllegalStateException("Stack is empty");
35         }
36         return stack[top];
37     }
38
39     public boolean isEmpty() {
40         return top == -1;
41     }
42 }
```

```

42
43     public int size() {
44         return top + 1;
45     }
46
47     @Override
48     public Iterator<T> iterator() {
49         return new StackIterator();
50     }
51
52     private class StackIterator implements Iterator<T> {
53         private int current = top;
54
55         @Override
56         public boolean hasNext() {
57             return current >= 0;
58         }
59
60         @Override
61         public T next() {
62             if (!hasNext()) {
63                 throw new NoSuchElementException();
64             }
65             return stack[current--];
66         }
67     }
68 }

```

Listing 6: Implementación del iterador para StackGeneric en Java

```

1 public class StackGenericTest {
2     public static void main(String[] args) {
3         StackGeneric<Integer> stack = new StackGeneric<>(5);
4         stack.push(1);
5         stack.push(2);
6         stack.push(3);
7
8         for (Integer element : stack) {
9             System.out.println(element);
10        }
11    }
12 }

```

Listing 7: Ejemplo de uso del iterador para StackGeneric en Java

1.8.2. Iteradores en Python

En Python, los iteradores se implementan utilizando el método `__iter__()` y el protocolo de iteración. El método `__iter__()` debe retornar un objeto iterador, que a su vez debe implementar los métodos `__next__()` y `__iter__()`. El método `__next__()` retorna el siguiente elemento de la colección, y el método `__iter__()` retorna el propio objeto iterador.

A continuación, se presenta una implementación más idiomática en Python, utilizando un generador (`yield`) para la iteración. Esto elimina la necesidad de una clase `StackIterator` explícita, resultando en un código más limpio y conciso.

```

1 class Stack:
2     def __init__(self):

```

```

3     self.items = []
4
5     def push(self, item):
6         self.items.append(item)
7
8     def pop(self):
9         if not self.is_empty():
10            return self.items.pop()
11        return None
12
13    def peek(self):
14        if not self.is_empty():
15            return self.items[-1]
16        return None
17
18    def is_empty(self):
19        return len(self.items) == 0
20
21    def size(self):
22        return len(self.items)
23
24    def __iter__(self):
25        """
26        Iterador que recorre la pila desde la cima (LIFO).
27        """
28        for i in range(len(self.items) - 1, -1, -1):
29            yield self.items[i]

```

Listing 8: Implementación del iterador para Stack en Python con yield

```

1 stack = Stack()
2 stack.push(1)
3 stack.push(2)
4 stack.push(3)
5
6 for item in stack:
7     print(item)

```

Listing 9: Ejemplo de uso del iterador para Stack en Python

1.8.3. Iteradores en C#

En C#, los iteradores se implementan utilizando la interfaz `IEnumerable` y `IEnumerator`. La interfaz `IEnumerable` define el método `GetEnumerator()`, que retorna un objeto de tipo `IEnumerator`. La interfaz `IEnumerator` define los métodos `MoveNext()`, que avanza al siguiente elemento de la colección, `Current`, que retorna el elemento actual de la colección, y `Reset()`, que reinicia el iterador al inicio de la colección.

A diferencia de Java, en C# es necesario implementar dos interfaces: la versión genérica `IEnumerable<T>` (para el uso moderno con tipos seguros) y la versión no genérica `IEnumerable` (para compatibilidad con código anterior que no usa genéricos). Si no se incluye la versión no genérica, el compilador genera un error. A continuación, se presenta un ejemplo de implementación:

```

1 using System;
2 using System.Collections;
3 using System.Collections.Generic;
4
5 public class StackGeneric<T> : IEnumerable<T>

```

```
6 {
7     private T[] stack;
8     private int top;
9     private int capacity;
10
11     public StackGeneric(int capacity)
12     {
13         this.capacity = capacity;
14         this.stack = new T[capacity];
15         this.top = -1;
16     }
17
18     public void Push(T element)
19     {
20         if (top == capacity - 1)
21         {
22             throw new InvalidOperationException("Stack is full");
23         }
24         stack[++top] = element;
25     }
26
27     public T Pop()
28     {
29         if (IsEmpty())
30         {
31             throw new InvalidOperationException("Stack is empty");
32         }
33         return stack[top--];
34     }
35
36     public T Peek()
37     {
38         if (IsEmpty())
39         {
40             throw new InvalidOperationException("Stack is empty");
41         }
42         return stack[top];
43     }
44
45     public bool IsEmpty()
46     {
47         return top == -1;
48     }
49
50     public int Size()
51     {
52         return top + 1;
53     }
54
55     public IEnumerator<T> GetEnumerator()
56     {
57         return new StackEnumerator(this);
58     }
59
60     IEnumerator IEnumerable.GetEnumerator()
61     {
62         return GetEnumerator();
63     }
64 }
```

```

65 private class StackEnumerator : IEnumerator<T>
66 {
67     private StackGeneric<T> stack;
68     private int current;
69
70     public StackEnumerator(StackGeneric<T> stack)
71     {
72         this.stack = stack;
73         // El iterador se posiciona antes del primer elemento.
74         this.current = stack.top + 1;
75     }
76
77     public bool MoveNext()
78     {
79         // Avanza al siguiente elemento y retorna true si hay mas elementos.
80         current--;
81         return current >= 0;
82     }
83
84     public void Reset()
85     {
86         current = stack.top + 1;
87     }
88
89     public T Current
90     {
91         get
92         {
93             if (current < 0 || current > stack.top)
94             {
95                 throw new InvalidOperationException();
96             }
97             return stack.stack[current];
98         }
99     }
100
101     object IEnumerator.Current
102     {
103         get { return Current; }
104     }
105
106     public void Dispose()
107     {
108         // No resources to dispose
109     }
110 }
111 }

```

Listing 10: Implementación del iterador para StackGeneric en C#

```

1 public class StackGenericTest
2 {
3     public static void Main(string[] args)
4     {
5         StackGeneric<int> stack = new StackGeneric<int>(5);
6         stack.Push(1);
7         stack.Push(2);
8         stack.Push(3);
9     }

```

```

10     foreach (int element in stack)
11     {
12         Console.WriteLine(element);
13     }
14 }
15 }

```

Listing 11: Ejemplo de uso del iterador para StackGeneric en C#

¿Por qué se usan dos interfaces en C#?

- La versión pública y fuertemente tipada (`IEnumerator<T>`) se usa cuando el consumidor trabaja con la clase a través de `IEnumerable<T>`, obteniendo elementos del tipo correcto (T) sin necesidad de hacer casting. Esta versión es la forma moderna de usar enumeradores.
- La versión explícita no genérica (`IEnumerator`) existe para cumplir con el contrato de la vieja interfaz `IEnumerable` y garantizar compatibilidad con código que no usa genéricos (por ejemplo, `foreach` sobre una referencia de tipo `IEnumerable` en lugar de `IEnumerable<T>`). Si no se incluye, genera un error de compilación.

1.9. Implementación de la Cola con Arreglo Circular

Una implementación simple de una cola (Queue) usando un arreglo implicaría que al hacer `dequeue`, todos los elementos restantes deberían moverse una posición hacia adelante, lo cual es una operación costosa de $O(n)$. Para evitar esto, se utiliza una técnica llamada **arreglo circular**.

En un arreglo circular, mantenemos dos punteros: **head** (cabeza) y **tail** (cola). **head** apunta al primer elemento y **tail** apunta a la siguiente posición libre. Cuando un puntero llega al final del arreglo, vuelve al principio (de ahí el nombre circular). Esto se logra fácilmente con el operador módulo (

```

1  import java.util.Iterator;
2  import java.util.NoSuchElementException;
3
4  public class QueueCircularArray<T> implements Iterable<T> {
5      private T[] queue;
6      private int head; // Puntero al primer elemento
7      private int tail; // Puntero a la proxima posicion libre
8      private int n;    // Numero de elementos en la cola
9
10     public QueueCircularArray() {
11         queue = (T[]) new Object[2]; // Empezar con capacidad 2
12         head = 0;
13         tail = 0;
14         n = 0;
15     }
16
17     public boolean isEmpty() {
18         return n == 0;
19     }
20
21     public int size() {
22         return n;
23     }
24
25     private void resize(int capacity) {
26         assert capacity >= n;
27         T[] copy = (T[]) new Object[capacity];

```

```
28     for (int i = 0; i < n; i++) {
29         copy[i] = queue[(head + i) % queue.length];
30     }
31     queue = copy;
32     head = 0;
33     tail = n;
34 }
35
36 public void enqueue(T element) {
37     if (n == queue.length) {
38         resize(2 * queue.length); // Doblar el tamaño si está lleno
39     }
40     queue[tail] = element;
41     tail = (tail + 1) % queue.length; // Avanzar y dar la vuelta si es
necesario
42     n++;
43 }
44
45 public T dequeue() {
46     if (isEmpty()) {
47         throw new IllegalStateException("Queue is empty");
48     }
49     T element = queue[head];
50     queue[head] = null; // Evitar loitering
51     head = (head + 1) % queue.length; // Avanzar y dar la vuelta si es
necesario
52     n--;
53     if (n > 0 && n == queue.length / 4) {
54         resize(queue.length / 2); // Reducir a la mitad si está a 1/4 de
capacidad
55     }
56     return element;
57 }
58
59 public T peek() {
60     if (isEmpty()) {
61         throw new IllegalStateException("Queue is empty");
62     }
63     return queue[head];
64 }
65
66 @Override
67 public Iterator<T> iterator() {
68     return new QueueIterator();
69 }
70
71 private class QueueIterator implements Iterator<T> {
72     private int current = 0;
73
74     @Override
75     public boolean hasNext() {
76         return current < n;
77     }
78
79     @Override
80     public T next() {
81         if (!hasNext()) {
82             throw new NoSuchElementException();
83         }
84     }
85 }
```



```

84         return queue[(head + current++) % queue.length];
85     }
86 }
87 }

```

Listing 12: Implementación de una Cola con arreglo circular en Java

Nota: Esta implementación de la cola incluye todas las características deseables: redimensionamiento dinámico para ajustar la capacidad según la necesidad, uso del arreglo circular para operaciones eficientes en tiempo constante $O(1)$ amortizado, y soporte para iteración mediante la interfaz `Iterable`.

1.10. Implementación de la Bolsa con Arreglo

La bolsa (Bag) es la más simple de las tres estructuras en términos de requisitos de orden. Como no hay un orden que preservar, la operación `add` simplemente puede agregar el elemento al final del arreglo, usando una estrategia de redimensionamiento dinámico igual que la pila.

La operación `remove(element)` es más compleja, ya que requiere encontrar el elemento en el arreglo. Una vez encontrado, no necesitamos mantener el orden, por lo que la forma más eficiente de eliminarlo es intercambiarlo con el último elemento del arreglo y luego eliminar el último elemento. Esto evita el costo de $O(n)$ de desplazar todos los elementos subsiguientes.

```

1  import java.util.Iterator;
2  import java.util.NoSuchElementException;
3
4  public class BagArray<T> implements Iterable<T> {
5      private T[] bag;
6      private int n; // Numero de elementos
7
8      public BagArray() {
9          bag = (T[]) new Object[2];
10         n = 0;
11     }
12
13     public boolean isEmpty() {
14         return n == 0;
15     }
16
17     public int size() {
18         return n;
19     }
20
21     private void resize(int capacity) {
22         T[] copy = (T[]) new Object[capacity];
23         for (int i = 0; i < n; i++) {
24             copy[i] = bag[i];
25         }
26         bag = copy;
27     }
28
29     public void add(T element) {
30         if (n == bag.length) {
31             resize(2 * bag.length);
32         }
33         bag[n++] = element;
34     }
35 }

```

```

36     public boolean remove(T element) {
37         for (int i = 0; i < n; i++) {
38             if (element.equals(bag[i])) {
39                 // Encontrado, intercambiar con el ultimo elemento
40                 bag[i] = bag[n - 1];
41                 bag[n - 1] = null; // Evitar loitering
42                 n--;
43                 if (n > 0 && n == bag.length / 4) {
44                     resize(bag.length / 2);
45                 }
46                 return true;
47             }
48         }
49         return false; // No encontrado
50     }
51
52     public boolean contains(T element) {
53         for (int i = 0; i < n; i++) {
54             if (element.equals(bag[i])) {
55                 return true;
56             }
57         }
58         return false;
59     }
60
61     @Override
62     public Iterator<T> iterator() {
63         return new BagIterator();
64     }
65
66     private class BagIterator implements Iterator<T> {
67         private int current = 0;
68
69         @Override
70         public boolean hasNext() {
71             return current < n;
72         }
73
74         @Override
75         public T next() {
76             if (!hasNext()) {
77                 throw new NoSuchElementException();
78             }
79             return bag[current++];
80         }
81     }
82 }

```

Listing 13: Implementación de una Bolsa con arreglo dinámico en Java

Nota: Esta implementación de la bolsa incluye todas las operaciones de la API definida: `add()`, `remove()`, `contains()`, `isEmpty()`, `size()` e `iterator()`. Además, utiliza redimensionamiento dinámico para optimizar el uso de memoria y la técnica de intercambio con el último elemento para hacer la eliminación más eficiente.

Nota sobre implementaciones en Python y C#: Las implementaciones de la Cola y la Bolsa se presentaron únicamente en Java, pero las mismas técnicas (redimensionamiento dinámico, arreglo circular, iteradores) se aplican directamente a Python y C#, siguiendo los patrones mostrados en la sección de la Pila. Se sugiere como ejercicio implementar estas estructuras en el lenguaje

de su preferencia.

1.11. Resumen comparativo

La siguiente tabla resume las características principales de las tres estructuras básicas estudiadas:

Característica	Bolsa (Bag)	Pila (Stack)	Cola (Queue)
Patrón de acceso	Sin orden	LIFO	FIFO
Inserción	add (al final)	push (cima)	enqueue (final)
Eliminación	remove (buscar)	pop (cima)	dequeue (frente)
Inserción (amortizado)	$O(1)$	$O(1)$	$O(1)$
Eliminación (amortizado)	$O(n)$ buscar, $O(1)$ eliminar	$O(1)$	$O(1)$
Consulta sin eliminar	contains $O(n)$	peek $O(1)$	peek $O(1)$

Cuadro 1: Comparación de las tres estructuras básicas

1.12. De arreglos a listas enlazadas

Hasta este punto, todas las implementaciones han utilizado arreglos como mecanismo de almacenamiento subyacente. Los arreglos ofrecen acceso rápido por índice ($O(1)$), pero tienen limitaciones importantes: insertar o eliminar elementos en posiciones arbitrarias requiere desplazar los elementos restantes ($O(n)$), y el redimensionamiento, aunque amortizado, implica copiar todo el arreglo. Las **listas enlazadas** ofrecen una alternativa: utilizan nodos conectados por referencias, lo que permite insertar y eliminar elementos de forma eficiente sin necesidad de desplazar datos. En las siguientes secciones estudiaremos las listas enlazadas y las utilizaremos como base para implementar las estructuras que hemos visto.

2. Listas Simplemente Enlazadas

2.1. Definición recursiva de lista

Una lista enlazada puede definirse recursivamente de la siguiente manera:

- Caso base: Una lista vacía es una referencia nula (`null`).
- Caso recursivo: Una lista es un `Nodo` que contiene un dato y una referencia a una lista, la cual a su vez puede ser el siguiente `Nodo` o la lista vacía.

En otras palabras, una lista enlazada es una secuencia de nodos, donde cada nodo contiene un elemento de datos y un enlace (o referencia) al siguiente nodo en la secuencia. El último nodo en la lista tiene un enlace nulo, indicando el final de la lista.

2.2. Implementación de la lista simplemente enlazada

Una lista simplemente enlazada se compone de nodos, donde cada nodo contiene un dato y una referencia al siguiente nodo en la lista.

2.2.1. Definición de la clase `Nodo`

A continuación, se muestra un ejemplo de la definición de la clase `Nodo` en Java:

```
1 class Nodo<T> {  
2     T dato;  
3     Nodo<T> siguiente;  
4  
5     Nodo(T dato) {  
6         this.dato = dato;  
7         this.siguiente = null;  
8     }  
9 }
```

Listing 14: Clase `Nodo` en Java

2.2.2. Construcción de una lista paso a paso

Para agregar un elemento a la lista, se crea un nuevo nodo con el dato deseado y se enlaza al principio de la lista. El procedimiento general es:

1. Crear un nuevo nodo con el dato a insertar.
2. Hacer que la referencia `siguiente` del nuevo nodo apunte a la cabeza actual de la lista.
3. Actualizar la cabeza de la lista para que apunte al nuevo nodo.

Ejemplo: Construyamos una lista con los elementos 1, 2 y 3, añadiendo cada nodo a la cabeza. Inicialmente, la lista está vacía: `Nodo<Integer>primero = null;`

1. **Añadimos el nodo con el valor 1.** Este nodo se convierte en la cabeza de la lista, y su referencia siguiente es nula.

```
1 Nodo<Integer> primero = new Nodo<>(1);  
2 primero.siguiente = null;
```

2. **Añadimos el nodo con el valor 2.** Este nuevo nodo se convierte en la nueva cabeza, y su referencia siguiente apunta al nodo con valor 1.

```
1 Nodo<Integer> segundo = new Nodo<>(2);
2 segundo.siguiente = primero;
3 primero = segundo;
```

3. **Añadimos el nodo con el valor 3.** Este nodo se convierte en la nueva cabeza, y su referencia siguiente apunta al nodo con valor 2.

```
1 Nodo<Integer> tercero = new Nodo<>(3);
2 tercero.siguiente = primero;
3 primero = tercero;
```

El resultado final es una lista enlazada donde el primer nodo contiene el valor 3, el segundo contiene el valor 2, y el tercero contiene el valor 1. Nótese que el orden de los nodos en la lista es inverso al orden de inserción, ya que siempre se agrega al principio.

2.2.3. Implementación de la lista enlazada como ADT

A continuación, se muestra un ejemplo de la implementación de la lista enlazada como un Abstract Data Type (ADT) en Java, con los métodos `addFirst`, `isEmpty`, y `size`.

Nota: En esta implementación, la clase `Nodo` se define como una **clase interna privada** de `ListaEnlazada`. A diferencia de la definición independiente mostrada anteriormente, esta versión no necesita su propio parámetro genérico `<T>` porque lo hereda de la clase contenedora. Definir `Nodo` como clase interna es la práctica recomendada, ya que impide que código externo manipule los nodos directamente, protegiendo la integridad de la estructura.

```
1 public class ListaEnlazada<T> {
2     private Nodo cabeza;
3     private int size;
4
5     private class Nodo {
6         T dato;
7         Nodo siguiente;
8
9         Nodo(T dato) {
10             this.dato = dato;
11             this.siguiente = null;
12         }
13     }
14
15     public ListaEnlazada() {
16         this.cabeza = null;
17         this.size = 0;
18     }
19
20     public void addFirst(T dato) {
21         Nodo nuevoNodo = new Nodo(dato);
22         nuevoNodo.siguiente = cabeza;
23         cabeza = nuevoNodo;
24         size++;
25     }
26
27     public boolean isEmpty() {
28         return cabeza == null;
29     }
30 }
```

```

29     }
30
31     public int size() {
32         return size;
33     }
34 }

```

Listing 15: Lista Enlazada como ADT en Java

Ejemplo en Python: Implementación de la lista enlazada como ADT, con métodos: `add_first`, `is_empty`, `size`.

```

1 class Nodo:
2     def __init__(self, dato):
3         self.dato = dato
4         self.siguiente = None
5
6 class ListaEnlazada:
7     def __init__(self):
8         self.cabeza = None
9         self._size = 0
10
11     def add_first(self, dato):
12         nuevo_nodo = Nodo(dato)
13         nuevo_nodo.siguiente = self.cabeza
14         self.cabeza = nuevo_nodo
15         self._size += 1
16
17     def is_empty(self):
18         return self.cabeza is None
19
20     def size(self):
21         return self._size

```

Listing 16: Lista Enlazada como ADT en Python

Ejemplo en C#: Implementación de la lista enlazada como ADT, con métodos: `AddFirst`, `IsEmpty`, `Size`.

Nota sobre convenciones de nomenclatura: En los ejemplos en C#, se siguen las convenciones del lenguaje, usando nombres en inglés (`LinkedList`, `Node`) en lugar de los nombres en español (`ListaEnlazada`, `Nodo`) usados en Java y Python. Las implementaciones son equivalentes en funcionalidad.

```

1 public class Node<T>
2 {
3     public T Data { get; set; }
4     public Node<T> Next { get; set; }
5
6     public Node(T data)
7     {
8         Data = data;
9         Next = null;
10    }
11 }
12
13 public class LinkedList<T>
14 {
15     private Node<T> head;
16     private int size;

```

```

17
18     public LinkedList()
19     {
20         head = null;
21         size = 0;
22     }
23
24     public void AddFirst(T data)
25     {
26         Node<T> newNode = new Node<T>(data);
27         newNode.Next = head;
28         head = newNode;
29         size++;
30     }
31
32     public bool IsEmpty()
33     {
34         return head == null;
35     }
36
37     public int Size()
38     {
39         return size;
40     }
41 }

```

Listing 17: Lista Enlazada como ADT en C#

2.3. Otras operaciones con listas simplemente enlazadas

Además de las operaciones básicas de agregar elementos y verificar si la lista está vacía, se pueden implementar otras operaciones útiles en listas simplemente enlazadas. A continuación, se describen algunas de estas operaciones, junto con su firma y una explicación conceptual de su implementación.

2.3.1. Eliminar el primer elemento de la lista

Lenguaje	Firma del método removeFirst()
Java	public T removeFirst()
Python	def remove_first(self):
C#	public T RemoveFirst()

Cuadro 2: Firma del método removeFirst() en diferentes lenguajes

Implementación conceptual:

1. Verificar si la lista está vacía. Si lo está, lanzar una excepción.
2. Guardar el dato del primer nodo.
3. Actualizar la cabeza de la lista para que apunte al segundo nodo.
4. Disminuir el tamaño de la lista en 1.
5. Retornar el dato guardado.

2.3.2. Agregar un elemento al final de la lista

Lenguaje	Firma del método addLast()
Java	public void addLast(T dato)
Python	def add_last(self, dato):
C#	public void AddLast(T data)

Cuadro 3: Firma del método addLast en diferentes lenguajes

Implementación conceptual:

1. Crear un nuevo nodo con el dato a insertar.
2. Verificar si la lista está vacía. Si lo está, el nuevo nodo se convierte en la cabeza de la lista.
3. Si la lista no está vacía, recorrer la lista hasta llegar al último nodo.
4. Hacer que la referencia siguiente del último nodo apunte al nuevo nodo.
5. Aumentar el tamaño de la lista en 1.

2.3.3. Eliminar el último elemento de la lista

Lenguaje	Firma del método removeLast()
Java	public T removeLast()
Python	def remove_last(self):
C#	public T RemoveLast()

Cuadro 4: Firma del método removeLast en diferentes lenguajes

Implementación conceptual:

1. Verificar si la lista está vacía. Si lo está, no se puede eliminar ningún elemento.
2. Si la lista contiene un solo elemento, establecer la cabeza de la lista a `null`.
3. Si la lista contiene más de un elemento, recorrer la lista hasta llegar al penúltimo nodo.
4. Hacer que la referencia siguiente del penúltimo nodo sea `null`.
5. Disminuir el tamaño de la lista en 1.

Ejemplo Java: Implementación de las operaciones eliminar primer elemento y agregar al final.

```

1 public class ListaEnlazada<T> {
2     private Nodo cabeza;
3     private int size;
4
5     public ListaEnlazada() {
6         this.cabeza = null;
7         this.size = 0;
8     }
9
10    public void addFirst(T dato) {

```



```

11     Nodo nuevoNodo = new Nodo(dato);
12     nuevoNodo.siguiente = cabeza;
13     cabeza = nuevoNodo;
14     size++;
15 }
16
17 public void addLast(T dato) {
18     Nodo nuevoNodo = new Nodo(dato);
19     if (isEmpty()) {
20         cabeza = nuevoNodo;
21     } else {
22         Nodo current = cabeza;
23         while (current.siguiente != null) {
24             current = current.siguiente;
25         }
26         current.siguiente = nuevoNodo;
27     }
28     size++;
29 }
30
31 public T removeFirst() {
32     if (isEmpty()) {
33         throw new java.util.NoSuchElementException();
34     }
35     T dato = cabeza.dato;
36     cabeza = cabeza.siguiente;
37     size--;
38     return dato;
39 }
40
41 public int size() {
42     return size;
43 }
44
45 public boolean isEmpty() {
46     return cabeza == null;
47 }
48 }

```

Listing 18: Operaciones en Lista Enlazada en Java

2.3.4. Otras operaciones con listas simples

La siguiente tabla describe algunas operaciones adicionales que frecuentemente se implementan en listas simples:

Operación	Firma de la operación
Obtener dato en posición	<code>T get(int index)</code>
Asignar dato en posición	<code>void set(int index, T data)</code>
Remover el dato en una posición	<code>void remove(int index)</code>
Verificar si un dato está en la lista	<code>boolean contains(T data)</code>
Limpiar toda la lista	<code>void clear()</code>

Cuadro 5: Otras operaciones con listas

Ejercicio: Implementar las operaciones descritas en la tabla anterior. Considere las siguientes

preguntas para guiar su implementación:

- Para `get(index)` y `set(index, data)`: ¿cómo se puede llegar al nodo en la posición indicada? ¿Cuál es la complejidad temporal?
- Para `remove(index)`: ¿qué sucede si el índice corresponde al primer elemento? ¿Y al último? ¿Qué nodo necesita modificarse?
- Para `contains(data)`: ¿se puede reutilizar el patrón de recorrido secuencial?
- Para `clear()`: ¿es suficiente con establecer la cabeza como `null`? ¿Qué pasa con el campo `size`?

2.4. Iteradores sobre listas simplemente enlazadas

El recorrido de una lista enlazada es un proceso fundamental para realizar diversas operaciones sobre sus elementos. Un iterador es un objeto que permite recorrer una lista (u otra estructura de datos) y acceder a sus elementos de manera secuencial, sin exponer la estructura interna de la lista.

2.4.1. Proceso de recorrido de una lista

Para recorrer una lista enlazada, se utiliza un puntero (o referencia) que inicialmente apunta a la cabeza de la lista. Luego, se itera sobre la lista, moviendo el puntero al siguiente nodo en cada paso, hasta que el puntero llegue al final de la lista (es decir, apunte a `null`).

2.4.2. Operaciones que dependen de un recorrido de la lista

Muchas operaciones comunes en listas enlazadas requieren un recorrido de la lista. Algunas de estas operaciones son:

- **Recorrer una lista:** Visitar cada nodo de la lista y realizar alguna acción sobre su dato (por ejemplo, imprimirlo).
- **Buscar un elemento en una lista:** Recorrer la lista hasta encontrar un nodo cuyo dato coincida con el valor buscado.
- **Eliminar un elemento arbitrario de una lista:** Recorrer la lista hasta encontrar el nodo que se desea eliminar, y luego actualizar las referencias de los nodos adyacentes para eliminar el nodo de la lista.
- **Insertar un elemento en una posición arbitraria de una lista:** Recorrer la lista hasta encontrar la posición donde se desea insertar el nuevo nodo, y luego actualizar las referencias de los nodos adyacentes para insertar el nuevo nodo en la lista.

Ejemplos Java:

- Implementación del iterador de listas.
- Implementación de la búsqueda secuencial en la lista.

```

1 import java.util.Iterator;
2
3 public class ListaEnlazada<T> implements Iterable<T> {
4     private Nodo cabeza;
5     private int size;
6
7     // Constructor, addFirst, addLast, removeFirst, size, isEmpty (como antes)
8
9     @Override
10    public Iterator<T> iterator() {
11        return new IteradorListaEnlazada();
12    }
13
14    private class IteradorListaEnlazada implements Iterator<T> {
15        private Nodo current = cabeza;
16
17        @Override
18        public boolean hasNext() {
19            return current != null;
20        }
21
22        @Override
23        public T next() {
24            if (!hasNext()) {
25                throw new java.util.NoSuchElementException();
26            }
27            T dato = current.dato;
28            current = current.siguiente;
29            return dato;
30        }
31    }
32 }

```

Listing 19: Iterador de Listas en Java

```

1 public class ListaEnlazada<T> {
2     // ... (codigo anterior)
3
4     public boolean buscar(T valor) {
5         Nodo current = cabeza;
6         while (current != null) {
7             if (current.dato.equals(valor)) {
8                 return true;
9             }
10            current = current.siguiente;
11        }
12        return false;
13    }
14 }

```

Listing 20: Búsqueda Secuencial en Java

Ejemplos Python:

- Implementación del iterador de listas.
- Implementación de la búsqueda secuencial en la lista.

```

1 class Nodo:
2     def __init__(self, dato):
3         self.dato = dato
4         self.siguiente = None
5
6 class ListaEnlazada:
7     def __init__(self):
8         self.cabeza = None
9         self._size = 0
10
11     # add, isEmpty, size (como antes)
12
13     def __iter__(self):
14         current = self.cabeza
15         while current is not None:
16             yield current.dato
17             current = current.siguiente

```

Listing 21: Iterador de Listas en Python

```

1 class ListaEnlazada:
2     # ... (codigo anterior)
3
4     def buscar(self, valor):
5         current = self.cabeza
6         while current is not None:
7             if current.dato == valor:
8                 return True
9             current = current.siguiente
10        return False

```

Listing 22: Búsqueda Secuencial en Python

Ejemplos C#:

- Implementación del iterador de listas.
- Implementación de la búsqueda secuencial en la lista.

```

1 using System.Collections;
2 using System.Collections.Generic;
3
4 public class LinkedList<T> : IEnumerable<T>
5 {
6     private Node<T> head;
7     private int size;
8
9     // Constructor, AddFirst, AddLast, RemoveFirst, Size, IsEmpty (como antes)
10
11     public IEnumerator<T> GetEnumerator()
12     {
13         Node<T> current = head;
14         while (current != null)
15         {
16             yield return current.Data;
17             current = current.Next;
18         }
19     }
20

```

```

21     IEnumerator IEnumerable.GetEnumerator()
22     {
23         return GetEnumerator();
24     }
25 }

```

Listing 23: Iterador de Listas en C#

```

1 public class LinkedList<T>
2 {
3     // ... (codigo anterior)
4
5     public bool Contains(T valor)
6     {
7         Node<T> current = head;
8         while (current != null)
9         {
10             if (current.Data.Equals(valor))
11             {
12                 return true;
13             }
14             current = current.Next;
15         }
16         return false;
17     }
18 }

```

Listing 24: Búsqueda Secuencial en C#

2.5. Resumen de complejidad de operaciones

La siguiente tabla resume la complejidad temporal de las operaciones principales en una lista simplemente enlazada (sin puntero al último nodo):

Operación	Complejidad	Razón
addFirst	$O(1)$	Inserción directa en la cabeza
addLast	$O(n)$	Requiere recorrer toda la lista
removeFirst	$O(1)$	Actualización directa de la cabeza
removeLast	$O(n)$	Requiere encontrar el penúltimo nodo
get(index)	$O(n)$	Requiere recorrer hasta la posición
contains(data)	$O(n)$	Requiere recorrido secuencial

Cuadro 6: Complejidad temporal de operaciones en lista simplemente enlazada

Nótese que `addLast` podría mejorarse a $O(1)$ si se mantuviera un puntero adicional al último nodo (*tail pointer*), a costa de una mayor complejidad en la implementación.

3. Listas Doblemente Enlazadas

En la lista simplemente enlazada, algunas operaciones son inherentemente costosas. Por ejemplo, `removeLast()` requiere recorrer toda la lista para encontrar el penúltimo nodo, ya que desde el último nodo no se puede retroceder. De manera similar, eliminar un nodo arbitrario requiere recorrer la lista desde la cabeza para encontrar el nodo anterior. Una **lista doblemente enlazada** resuelve

estos problemas al agregar un enlace adicional en cada nodo: además de la referencia al nodo siguiente, cada nodo mantiene una referencia al nodo anterior. Esto permite recorrer la lista en ambas direcciones y realizar operaciones como `removeLast()` en $O(1)$.

3.1. ADT e Implementación de la lista doblemente enlazada

El ADT de una lista doblemente enlazada incluye operaciones como:

- `addFirst(T data)`: Agrega un elemento al principio de la lista.
- `addLast(T data)`: Agrega un elemento al final de la lista.
- `removeFirst()`: Elimina el primer elemento de la lista.
- `removeLast()`: Elimina el último elemento de la lista.
- `getFirst()`: Obtiene el primer elemento de la lista.
- `getLast()`: Obtiene el último elemento de la lista.
- `size()`: Devuelve el número de elementos en la lista.
- `isEmpty()`: Indica si la lista está vacía.

A continuación, se muestra un ejemplo de implementación básica de una lista doblemente enlazada en Java:

```
1 import java.util.NoSuchElementException;
2
3 public class ListaDoblementeEnlazada<T> {
4     private NodoDoble cabeza;
5     private NodoDoble cola;
6     private int size;
7
8     private class NodoDoble {
9         T dato;
10        NodoDoble siguiente;
11        NodoDoble anterior;
12
13        NodoDoble(T dato) {
14            this.dato = dato;
15            this.siguiente = null;
16            this.anterior = null;
17        }
18    }
19
20    public ListaDoblementeEnlazada() {
21        this.cabeza = null;
22        this.cola = null;
23        this.size = 0;
24    }
25
26    public void addFirst(T dato) {
27        NodoDoble nuevoNodo = new NodoDoble(dato);
28        if (isEmpty()) {
29            cabeza = nuevoNodo;
30            cola = nuevoNodo;
```

```
31     } else {
32         nuevoNodo.siguiente = cabeza;
33         cabeza.anterior = nuevoNodo;
34         cabeza = nuevoNodo;
35     }
36     size++;
37 }
38
39 public void addLast(T dato) {
40     NodoDoble nuevoNodo = new NodoDoble(dato);
41     if (isEmpty()) {
42         cabeza = nuevoNodo;
43         cola = nuevoNodo;
44     } else {
45         nuevoNodo.anterior = cola;
46         cola.siguiente = nuevoNodo;
47         cola = nuevoNodo;
48     }
49     size++;
50 }
51
52 public T getFirst() {
53     if (isEmpty()) {
54         throw new NoSuchElementException("La lista esta vacia");
55     }
56     return cabeza.dato;
57 }
58
59 public T getLast() {
60     if (isEmpty()) {
61         throw new NoSuchElementException("La lista esta vacia");
62     }
63     return cola.dato;
64 }
65
66
67 public T removeFirst() {
68     if (isEmpty()) {
69         throw new NoSuchElementException("La lista esta vacia");
70     }
71     T dato = cabeza.dato;
72     if (cabeza == cola) {
73         cabeza = null;
74         cola = null;
75     } else {
76         cabeza = cabeza.siguiente;
77         cabeza.anterior = null;
78     }
79     size--;
80     return dato;
81 }
82
83 public T removeLast() {
84     if (isEmpty()) {
85         throw new NoSuchElementException("La lista esta vacia");
86     }
87     T dato = cola.dato;
88     if (cabeza == cola) {
89         cabeza = null;
```

```
90         cola = null;
91     } else {
92         cola = cola.anterior;
93         cola.siguiente = null;
94     }
95     size--;
96     return dato;
97 }
98
99
100 public int size() {
101     return size;
102 }
103
104 public boolean isEmpty() {
105     return size == 0;
106 }
107 }
```

Listing 25: Lista Doblemente Enlazada en Java

Ejercicio: Implementar la lista doblemente enlazada en Python y/o C#, siguiendo los patrones mostrados en la sección de listas simplemente enlazadas. Considere cómo la estructura del nodo doble (*anterior* + *siguiente*) afecta cada operación.

3.2. La cola de doble terminación (DEQUE)

Una cola de doble terminación (Double-Ended Queue (DEQUE)) es una generalización de una cola en la que se pueden insertar y eliminar elementos tanto al principio como al final de la estructura. Es decir, un DEQUE puede funcionar como una cola (FIFO) o como una pila (LIFO).

El ADT de un DEQUE incluye operaciones como:

- `addFirst(T data)`: Agrega un elemento al principio del DEQUE.
- `addLast(T data)`: Agrega un elemento al final del DEQUE.
- `removeFirst()`: Elimina el primer elemento del DEQUE.
- `removeLast()`: Elimina el último elemento del DEQUE.
- `getFirst()`: Obtiene el primer elemento del DEQUE.
- `getLast()`: Obtiene el último elemento del DEQUE.
- `size()`: Devuelve el número de elementos en el DEQUE.
- `isEmpty()`: Indica si el DEQUE está vacío.

Observando el API del DEQUE, se puede notar que es idéntico al de la lista doblemente enlazada. En efecto, una lista doblemente enlazada es una implementación natural del DEQUE. La diferencia es conceptual: el DEQUE es una *abstracción* (un ADT) que define un contrato de operaciones, mientras que la lista doblemente enlazada es una *implementación* concreta. Un DEQUE también podría implementarse usando un arreglo circular, como se vio en la sección anterior con la cola.

Ejercicio: Implementar el DEQUE utilizando la lista doblemente enlazada presentada anteriormente. Considere:

- ¿Qué métodos de `ListaDoblementeEnlazada` corresponden directamente a las operaciones del DEQUE?
- ¿Es necesario agregar funcionalidad nueva, o basta con renombrar las operaciones existentes?
- ¿Cómo se implementaría un DEQUE basado en un arreglo circular? ¿Qué ventajas y desventajas tendría frente a la implementación con lista enlazada?

